



Swift

What is Swift?



- Swift is a powerful, modern programming language developed by Apple in 2014
- Used primarily for iOS, macOS, watchOS, and tvOS apps . *ipados & visionos*
- Safe, fast, and expressive – easier for beginners than older languages like Objective-C
- Open-source and cross-platform capable
- Key features: Type safety, optionals for nil handling, modern syntax

Why Learn Swift?



- High demand for iOS developers
- Build apps for iPhone, iPad, Mac, Apple Watch
- Clean and readable code – great for beginners
- Integrates perfectly with Apple's ecosystem
- Future-proof: SwiftUI is the modern way to build UIs

Features



- Modern & Expressive Syntax

- Clean and readable - Inspired by Python and Ruby
- Type inference - Compiler deduces types automatically
- No semicolons required (optional)
- Concise closures with trailing syntax

- Type Safety & Type Inference

- Statically typed - Types checked at compile-time
- Type inference minimizes verbosity
- Optionals handle absence of values explicitly ✖

- Memory Safety

- Automatic Reference Counting (ARC) - Manages memory automatically
- No garbage collection pauses
- Value semantics for many types prevent accidental sharing
- Prevents common memory errors (buffer overflows, use-after-free)

- Optionals

- Elegant handling of missing/nil values
- Forces explicit unwrapping - prevents null pointer exceptions
- Multiple unwrapping techniques

- Protocol-Oriented Programming

- Protocols define blueprints of functionality
- Protocol extensions provide default implementations
- Composition over inheritance

- Value Types

- Structures and Enums are value types
- Copy semantics - each instance is independent
- Safer concurrency - no unintended sharing

- Pattern Matching

- Powerful switch statements
- Tuple matching
- Value bindings
- Where clauses

- Pattern Matching

- Powerful switch statements
- Tuple matching
- Value bindings
- Where clauses

Advantages



- **Safe** - Prevents many common programming errors
- **Fast** - Performance comparable to C++
- **Expressive** - Clean, readable syntax
- **Modern** - Continuously updated with new features
- **Versatile** - From mobile apps to server-side development
- **Apple-optimized** - Perfect integration with Apple ecosystems
- **Growing ecosystem** - Strong community and open-source libraries

History



- **2010** - Development begins at Apple, led by Chris Lattner, as a secret project.
- **2014** - Swift 1.0 announced at WWDC as "Objective-C without the C."
- **2015** - Swift 2.0 introduces error handling, protocol extensions, and goes open-source.
- **2016** - Swift 3.0 brings major API redesign and Linux support.
- **2017** - Swift 4.0 adds Codable, string improvements, and source stability.
- **2018** - Swift 4.2 introduces case iterable and random number generation.
- **2019** - Swift 5.0 achieves ABI stability and adds raw strings.
- **2019** - SwiftUI announced, revolutionizing Apple's UI development.
- **2020** - Swift 5.3 adds multiple trailing closures and improved error messages.
- **2021** - Swift 5.5 introduces async/await and actors for concurrency.
- **2022** - Swift 5.7 brings regular expressions and existential any keyword.
- **2023** - Swift 5.9 adds macros and ownership features.
- **2024** - Swift 6 development begins with focus on data race safety.
- **Today** - Used by millions for iOS, macOS, server-side, and cross-platform development.

Setting Up Your Environment



- **Requirements:**

- Mac with macOS 10.15+
- Xcode (Free from App Store)
- Apple ID (Free developer account)

- **Two Ways to Learn:**

- Xcode Playgrounds → Quick experimentation
- Xcode Projects → Real iOS apps

Hello World in Swift



```
print("Hello, Swift World!")
```

- print() → Function that outputs text
- "Hello, Swift World! " → String literal
- No semicolon needed (optional)
- Case-sensitive → Print ≠ print

Comments



```
// Single-line comment – for short notes
```

```
/*
```

```
Multi-line comment
```

```
For longer explanations
```

```
or temporary code disabling
```

```
*/
```

```
/// Documentation comment
```

```
/// Appears in Quick Help (Option-click)
```

```
/// Use for function documentation
```

■ Best Practice:

- Comment why, not what
- Keep comments updated
- Use documentation comments for public APIs

Variables & Constants



// Variable – CAN change

var score = 100

score = 95 // Allowed

// Constant – CANNOT change

let playerName = "Alex"

// playerName = "Jordan" // ERROR!

- Use let by default
- Use var only when values change
- Naming: camelCase (e.g., playerName, highScore)
- Cannot start with numbers

Constants



// Good Practice

let maximumLoginAttempts = 3

let defaultUsername = "quest"

let applicationName = "MyAwesomeApp"

■ Benefits

- Safety - Value won't change accidentally
- Clarity - Intent is clear to other developers
- Performance - Compiler can optimize better
- Thread Safety - No race conditions

// When to use `var`:

var currentScore = 0 // Changes during game

var isDownloading = false // Tracks state

var shoppingCart = [String]() // Items added/removed

Naming Conventions



```
// Camel Case (Standard)
let userName = "john_doe"
let maximumAttemptCount = 5
var isNetworkAvailable = true
```

```
// Pascal Case (Types only)
struct UserProfile { }    // Good ✓
class GameViewController { } // Good ✓
let userprofile = UserProfile() // Bad naming ✗
```

```
// Clear, Descriptive Names
let n = 5 // Bad – what does 'n' mean? ✗
let numberOfPlayers = 5 // Good – clear! ✓
```

```
// Constants in ALL_CAPS (Optional)
let MAX_CONNECTIONS = 10
let API_KEY = "abc123"
```

Printing & Debugging



```
// Basic Print
print("Starting application...")

// Multiple Items
let name = "Alice"
let age = 25
print("Name:", name, "Age:", age)
// Output: Name: Alice Age: 25

// String Interpolation in Print
print("User \ (name) is \ (age) years old")

// Debug Print (shows type info)
debugPrint(name) // "Alice" (with quotes)
```

```
// Separator and Terminator
print("One", "Two", "Three", separator: " - ", terminator: "!")
// Output: One - Two - Three!

// Dump for complex objects
let numbers = [1, 2, 3]
dump(numbers)
/*
  ▾ 3 elements
    - 0: 1
    - 1: 2
    - 2: 3
  */
```

Common Errors & Solutions



```
// ERROR 1: Type Mismatch
var score = "100"
// score = score + 5 // ERROR: Can't add Int to String
score = String(Int(score)! + 5) // FIXED
```

```
// ERROR 2: Using Variable Before Initialization
var temperature: Double
// print(temperature) // ERROR: Not initialized
temperature = 23.5 // Initialize first!
```

```
// ERROR 3: Changing Constants
let pi = 3.14
// pi = 3.14159 // ERROR: `let` cannot be changed
```

```
// ERROR 4: Wrong Case
let Message = "Hello" // Wrong convention
let message = "Hello" // Correct
```

```
// ERROR 5: Reserved Words
// let class = "Math" // ERROR: 'class' is reserved
let `class` = "Math" // FIX: Use backticks
let myClass = "Math" // BETTER: Prefix it
```



Data Types



Value Types - Integers



```
// Signed Integers (can be positive/negative/zero)
```

```
let int8: Int8 = 127           // 8-bit (-128 to 127)
```

```
let int16: Int16 = 32767      // 16-bit
```

```
let int32: Int32 = 2147483647 // 32-bit
```

```
let int64: Int64 = 9223372036854775807 // 64-bit
```

```
let int: Int = 42             // Platform-native (32/64-bit)
```

```
// Unsigned Integers (positive only)
```

```
let uint8: UInt8 = 255       // 8-bit (0 to 255)
```

```
let uint16: UInt16 = 65535   // 16-bit
```

```
let uint32: UInt32           // 32-bit
```

```
let uint64: UInt64           // 64-bit
```

```
let uint: UInt = 100         // Platform-native
```

```
// Special Integer Values
```

```
let maxInt = Int.max         // Maximum value
```

```
let minInt = Int.min         // Minimum value
```


Value Types - Floating-Point Numbers



```
let float: Float = 3.14159           // 32-bit (~6 decimal digits)
let double: Double = 3.14159265359   // 64-bit (~15 decimal digits)

// Scientific Notation
let scientific = 1.25e2              // 125.0 ( $1.25 \times 10^2$ )
let negativeExp = 1.25e-2           // 0.0125

// Hexadecimal Floats
let hexFloat = 0xC.3p0              // 12.1875

// Special Values
let infinity = Double.infinity
let nan = Double.nan                // Not a Number
```

Value Types - Booleans



```
let isActive: Bool = true  
let isCompleted = false    // Type inferred
```

```
// From expressions
```

```
let isGreater = 5 > 3        // true  
let isEmpty = "".isEmpty    // true
```

Value Types - Characters & Strings



```
// Character (single Unicode character)
```

```
let letter: Character = "A"
```

```
let heart: Character = "❤️"
```

```
let emoji: Character = "🚀"
```

```
// String
```

```
let singleLine = "Hello, World!"
```

```
let multiline = """
```

```
    Line 1
```

```
    Line 2
```

```
    Line 3
```

```
    """
```

```
// Raw Strings (Swift 5+)
```

```
let rawString = #"User said: "Hello!""# // Quotes don't need escaping
```

```
let interpolatedRaw = #"2 + 2 = \#(2+2)"#
```

Type Inference vs Annotation

↳ implicit var declaration

// Type Inference (Swift guesses)

let city = "London" // Inferred as String

let population = 8900000 // Inferred as Int

let isCapital = true // Inferred as Bool

↳ explicit var declaration

// Type Annotation (You specify)

var username: String

username = "SwiftPro"

var score: Int

score = 100

// Mixed - Annotation with initialization

let gravity: Double = 9.8



Collection Types



■ Array

```
var numbers: [Int] = [1, 2, 3]  
var names = ["Alice", "Bob"] // Type inferred  
var emptyArray: [String] = []  
var repeated = Array(repeating: 0, count: 5) // [0, 0, 0, 0, 0]
```

// 2D Arrays

```
var matrix: [[Int]] = [[1, 2], [3, 4]]
```

■ Sets

```
var uniqueNumbers: Set<Int> = [1, 2, 3, 3, 2] // Becomes [1, 2, 3]  
var colors: Set = ["Red", "Green", "Blue"] // Type inferred  
var emptySet = Set<String>()
```

Collection Types



■ Dictionaries

datatype of key
datatype of value

```
var scores: [String: Int] = ["Alice": 95, "Bob": 87]  
var capitals = ["France": "Paris", "Japan": "Tokyo"]  
var emptyDict: [Int: String] = [:]
```

■ Tuples

// Basic Tuple

```
let person = ("John", 30, true) // (String, Int, Bool)
```

// Named Tuple

```
let namedPerson = (name: "Jane", age: 25, isStudent: false)  
print(namedPerson.name) // "Jane"
```

// Decomposition

```
let (name, age, _) = person  
print(name) // "John"
```

// Single-element tuples need comma

```
let single = ("hello",) // Type is (String,)
```

Special Types



■ Optional

```
var optionalInt: Int? = 42
optionalInt = nil // Allowed

// Implicitly Unwrapped Optionals (Use cautiously!)
var forcedString: String! = "Hello"
// Automatically unwrapped when used

// Optional Chaining
let length = optionalString?.count // Returns Int?
```

■ Ranges

```
// Closed Range (includes both ends)
let closed = 1...10 // 1, 2, 3, ..., 10

// Half-Open Range (excludes upper bound)
let halfOpen = 1..<10 // 1, 2, 3, ..., 9

// One-sided Ranges
let fromStart = ...5 // ..., 3, 4, 5
let toEnd = 5... // 5, 6, 7, ...
let through = ...5 // Same as fromStart

// Practical use
let array = [10, 20, 30, 40]
let slice = array[1...2] // [20, 30]
```

Type Safety



```
var score = 95      // Inferred as Int
score = "Ninety"    // COMPILE ERROR: Cannot assign String to Int
```

```
let age: Int = 25
age = 25.5          // ERROR: Cannot assign Double to Int
```

- Catches errors early (at compile time)
- Makes code predictable
- Prevents runtime crashes

Type Conversion



```
// String to Int
let numberString = "42"
let number = Int(numberString) // Returns optional Int?
```

```
// Int to String
let score = 100
let scoreText = String(score) // "100"
let message = "Score: \$(score)" // String interpolation
```

```
// Double to Int (truncates decimal)
let pi = 3.14159
let intPi = Int(pi) // 3 (loses decimal part)
```

```
// Numeric conversions
let doubleScore = Double(score) // 100.0
```

Strings - The Basics



```
// Concatenation
let firstName = "John"
let lastName = "Doe"
let fullName = firstName + " " + lastName

// String Interpolation (Recommended)
let age = 30
let greeting = "Hello, \(firstName)! You are \(age) years old."

// Multi-line Strings
let poem = """
Roses are red,
Violets are blue,
Swift is awesome,
And so are you!
"""

// String Length
let name = "Swift"
let count = name.count // 5
```

Strings - Useful Methods



```
let message = "Hello, Swift!"
```

```
// Common Operations
```

```
let upper = message.uppercased() // "HELLO, SWIFT!"
```

```
let lower = message.lowercased() // "hello, swift!"
```

```
let hasPrefix = message.hasPrefix("Hello") // true
```

```
let hasSuffix = message.hasSuffix("!") // true
```

```
// Substrings
```

```
let startIndex = message.index(message.startIndex, offsetBy: 7)
```

```
let endIndex = message.index(message.endIndex, offsetBy: -1)
```

```
let substring = message[startIndex..
```

```
// Replacement
```

```
let newMessage = message.replacingOccurrences(of: "Swift", with: "World")
```

```
// "Hello, World!"
```

Numbers & Math Operation



```
let a = 10
```

```
let b = 3
```

```
// Basic Operations
```

```
let sum = a + b // 13
```

```
let difference = a - b // 7
```

```
let product = a * b // 30
```

```
let quotient = a / b // 3 (Int division)
```

```
let remainder = a % b // 1
```

```
// Compound Assignment
```

```
var counter = 5
```

```
counter += 1 // 6 (counter = counter + 1)
```

```
counter -= 2 // 4
```

```
counter *= 3 // 12
```

```
counter /= 2 // 6
```

```
// With Decimals
```

```
let price = 19.99
```

```
let quantity = 3
```

```
let total = price * Double(quantity) // Must convert!
```

Comparison Operators



```
let a = 10
```

```
let b = 5
```

```
// Equality
```

```
a == b // false
```

```
a != b // true
```

```
// Greater/Less Than
```

```
a > b // true
```

```
a < b // false
```

```
a >= b // true
```

```
a <= b // false
```

```
// String Comparison (alphabetical)
```

```
"apple" < "banana" // true
```

```
"Swift" == "swift" // false (case-sensitive!)
```

```
"Swift".lowercased() == "swift" // true
```

```
// Practical Use
```

```
let isLoggedIn = true
```

```
let isValidPassword = false
```

Logical Operators



```
let isStudent = true
```

```
let hasID = false
```

```
let age = 18
```

```
// AND (&&) - Both must be true
```

```
let canEnter = isStudent && hasID // false
```

```
// OR (||) - At least one true
```

```
let canApply = isStudent || (age >= 18) // true
```

```
// NOT (!) - Reverse boolean
```

```
let cannotEnter = !canEnter // true
```

```
// Complex Conditions
```

```
let discountEligible = (age < 21 || age > 65) && isStudent
```

```
// false
```

```
// Parentheses for clarity
```

```
let result = (true && false) || (true && true) // true
```



Functions



What is a Function?

- **Self-Contained Code Units**
- **Functions** are reusable blocks of code that perform specific tasks
- **Like a recipe** - defines steps, can be "called" multiple times
- **Promotes** code reuse, readability, and maintainability

Why Use Functions?



```
// Without functions – repetitive!
```

```
print("Welcome, Alice!")
```

```
print("Your balance is $1000")
```

```
print("Last login: 2024-03-15")
```

```
print("Welcome, Bob!")
```

```
print("Your balance is $2500")
```

```
print("Last login: 2024-03-14")
```

```
// With functions – DRY (Don't Repeat Yourself)!
```

```
func greetUser(name: String, balance: Double, lastLogin: String) {
```

```
    print("Welcome, \(name)!")
```

```
    print("Your balance is $\(balance)")
```

```
    print("Last login: \(lastLogin)")
```

```
}
```

```
greetUser(name: "Alice", balance: 1000, lastLogin: "2024-03-15")
```

```
greetUser(name: "Bob", balance: 2500, lastLogin: "2024-03-14")
```

Basic Function Syntax



```
// 1. Function with no parameters, no return value
```

```
func sayHello() {  
    print("Hello, World!")  
}
```

```
// 2. Function with parameters
```

```
func greet(name: String) {  
    print("Hello, \(name)!")  
}
```

```
// 3. Function with return value
```

```
func square(number: Int) -> Int {  
    return number * number  
}
```

```
// 4. Calling functions
```

```
sayHello()                // Prints: Hello, World!  
greet(name: "Alice")      // Prints: Hello, Alice!  
let result = square(number: 5) // Returns: 25
```

Function Parameters



// 1. Single parameter

```
func greet(person: String) {  
    print("Hello, \(person)!")  
}
```

// 2. Multiple parameters

```
func introduce(name: String, age: Int, city: String) {  
    print("I'm \(name), \(age) years old from \(city)")  
}
```

// 3. Parameters with default values

```
func welcome(user: String, greeting: String = "Hello") {  
    print("\(greeting), \(user)!")  
}
```

```
welcome(user: "Alice") // Uses default: "Hello, Alice!"
```

```
welcome(user: "Bob", greeting: "Hi") // Custom: "Hi, Bob!"
```

// 4. Calling with argument labels

```
introduce(name: "Charlie", age: 30, city: "New York")
```

Parameter Labels & Argument Names



// External label (for calling) vs Internal name (in function body)

```
func calculateArea(of rectangle: String, width: Double, height: Double) -> Double {  
    print("Calculating area for: \(rectangle)")  
    return width * height  
}
```

// Calling with external labels

```
let area = calculateArea(of: "Living Room", width: 10.5, height: 8.2)
```

// Omitting external label with underscore _

```
func multiply(_ a: Int, _ b: Int) -> Int {  
    return a * b  
}
```

```
let product = multiply(5, 3) // No labels needed!
```

// Different internal/external names

```
func convertTemperature(from celsius: Double) -> Double {  
    return (celsius * 9/5) + 32  
}
```

```
let fahrenheit = convertTemperature(from: 25) // Clear external label
```

Return Values



```
// 1. Single return value
func square(_ number: Int) -> Int {
    return number * number
}
```

```
// 2. Multiple return values using Tuple
func minMax(array: [Int]) -> (min: Int, max: Int) {
    var currentMin = array[0]
    var currentMax = array[0]
    for value in array {
        if value < currentMin { currentMin = value }
        if value > currentMax { currentMax = value }
    }
    return (currentMin, currentMax)
}
```

```
let bounds = minMax(array: [8, -6, 2, 109, 3, 71])
print("Min: \(bounds.min), Max: \(bounds.max)")
```

```
// 3. Optional return value
```

```
// 3. Optional return value
func findIndex(of value: String, in array: [String]) -> Int? {
    for (index, item) in array.enumerated() {
        if item == value { return index }
    }
    return nil
}
```

```
// 4. No return value (Void)
func logMessage(_ message: String) -> Void {
    print("[LOG]: \(message)")
}
// Same as: func logMessage(_ message: String) { ... }
```

Function Types



// 1. Function type syntax: (ParameterTypes) -> ReturnType

```
func add(_ a: Int, _ b: Int) -> Int {  
    return a + b  
}
```

```
func multiply(_ a: Int, _ b: Int) -> Int {  
    return a * b  
}
```

// 2. Assign functions to variables

```
var mathOperation: (Int, Int) -> Int = add  
let result1 = mathOperation(5, 3) // 8
```

```
mathOperation = multiply
```

```
let result2 = mathOperation(5, 3) // 15
```

// 3. Pass functions as parameters

```
func calculate(_ operation: (Int, Int) -> Int, _ a: Int, _ b: Int) -> Int {  
    return operation(a, b)  
}  
let sum = calculate(add, 10, 20) // 30  
let product = calculate(multiply, 10, 20) // 200
```

// 4. Return functions from functions

```
func chooseOperation(isAddition: Bool) -> (Int, Int) -> Int {  
    return isAddition ? add : multiply  
}  
let selectedOp = chooseOperation(isAddition: true)  
selectedOp(7, 8) // 15
```

Nested Functions



```
// Outer function
func chooseStepFunction(backward: Bool) -> (Int) -> Int {

    // Nested function 1
    func stepForward(_ input: Int) -> Int {
        return input + 1
    }

    // Nested function 2
    func stepBackward(_ input: Int) -> Int {
        return input - 1
    }

    // Return one of the nested functions
    return backward ? stepBackward : stepForward
}
```



Variadic Parameters - Accepting Zero or More Values

```
// 1. Basic variadic parameter
func calculateAverage(_ numbers: Double...) -> Double {
    var total: Double = 0
    for number in numbers {
        total += number
    }
    return total / Double(numbers.count)
}

let avg1 = calculateAverage(1, 2, 3, 4, 5)           // 3.0
let avg2 = calculateAverage(10.5, 20.5, 30.5)        // 20.5
let avg3 = calculateAverage()                        // NaN (handle carefully!)

// 2. Multiple parameters with variadic
func joinStrings(separator: String, _ strings: String...) -> String {
    return strings.join(separator: separator)
}

let result = joinStrings(separator: ", ", "Apple", "Banana", "Cherry")
// "Apple, Banana, Cherry"
```


Function Overloading - Multiple Functions with Same Name



```
// Overloading by parameter types
func process(value: Int) {
    print("Processing integer: \(value)")
}
```

```
func process(value: String) {
    print("Processing string: \(value)")
}
```

```
func process(value: Double) {
    print("Processing double: \(value)")
}
```

```
process(value: 42)          // "Processing integer: 42"
process(value: "Hello")    // "Processing string: Hello"
process(value: 3.14)       // "Processing double: 3.14"
```

```
// Overloading by number of parameters
func add(_ a: Int, _ b: Int) -> Int {
    return a + b
}
```

```
func add(_ a: Int, _ b: Int, _ c: Int) -> Int {
    return a + b + c
}
```

```
func add(_ a: Double, _ b: Double) -> Double {
    return a + b
}
```

Default Parameter Values



```
// Functions with default values
func createUser(name: String,
               age: Int = 18,
               isAdmin: Bool = false,
               country: String = "USA") {
    print("Creating user: \(name), \(age), admin: \(isAdmin), from \(country)")
}

// Different ways to call
createUser(name: "Alice")           // Uses all defaults
createUser(name: "Bob", age: 25)    // Partial defaults
createUser(name: "Charlie", isAdmin: true) // Skip middle defaults
createUser(name: "David", age: 30, country: "UK") // Mix and match
createUser(name: "Eve", age: 28, isAdmin: true, country: "Canada")

// Parameters with defaults must be at the end
func calculateTotal(price: Double, quantity: Int = 1, taxRate: Double = 0.08) -> Double {
    return price * Double(quantity) * (1 + taxRate)
}

let total1 = calculateTotal(price: 100)           // 108.0
let total2 = calculateTotal(price: 100, quantity: 2) // 216.0
let total3 = calculateTotal(price: 100, taxRate: 0.1) // 110.0
```

Recursive Functions



```
// 1. Factorial calculation (n!)
func factorial(_ n: Int) -> Int {
    if n <= 1 {
        return 1 // Base case (stops recursion)
    }
    return n * factorial(n - 1) // Recursive case
}
```

```
print(factorial(5)) // 120 (5 * 4 * 3 * 2 * 1)
```

```
// 2. Fibonacci sequence
func fibonacci(_ n: Int) -> Int {
    if n <= 0 { return 0 }
    if n == 1 { return 1 }
    return fibonacci(n - 1) + fibonacci(n - 2)
}
```

```
// 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...
for i in 0...9 {
    print(fibonacci(i), terminator: " ")
}
```

Higher-Order Functions



// 1. Function that takes function as parameter

```
func applyOperation(_ numbers: [Int], operation: (Int) -> Int) -> [Int] {  
    var result = [Int]()  
    for number in numbers {  
        result.append(operation(number))  
    }  
    return result  
}
```

```
let numbers = [1, 2, 3, 4, 5]
```

// Pass closure (anonymous function)

```
let squared = applyOperation(numbers) { $0 * $0 } // [1, 4, 9, 16, 25]  
let doubled = applyOperation(numbers) { $0 * 2 } // [2, 4, 6, 8, 10]
```

// 2. Function that returns a function

```
func makeMultiplier(factor: Int) -> (Int) -> Int {  
    return { number in  
        return number * factor  
    }  
}
```

```
let timesThree = makeMultiplier(factor: 3)  
print(timesThree(7)) // 21
```

// 3. Function composition

```
func compose<A, B, C>(_ f: @escaping (B) -> C,  
    _ g: @escaping (A) -> B) -> (A) -> C {  
    return { x in f(g(x)) }  
}
```

```
func addOne(_ x: Int) -> Int { return x + 1 }  
func square(_ x: Int) -> Int { return x * x }
```

```
let addThenSquare = compose(square, addOne)  
print(addThenSquare(3)) // 16 (3+1=4, 42=16)
```



Collections



What are Collections?

- **Collections** store multiple values in a single variable
- **Four main types** in Swift
 - **Arrays** - Ordered collections
 - **Sets** - Unordered unique collections
 - **Dictionaries** - Key-Value pairs
 - **Tuples** - Fixed-size grouped values
- **Why Use Collections?**

```
// Without collections – Messy!
```

```
let student1 = "Alice"
```

```
let student2 = "Bob"
```

```
let student3 = "Charlie"
```

```
// Hard to manage!
```

```
// With collections – Organized!
```

```
let students = ["Alice", "Bob", "Charlie"]
```

Arrays



- **Characteristics**
- Ordered (maintains insertion order)
- Zero-based indexing
- Can contain duplicates
- Mutable (if declared with var)

```
// Declaration
var numbers: [Int] = [1, 2, 3, 4, 5]
var names = ["Alice", "Bob", "Charlie"] // Type inferred
var emptyArray: [String] = []
var initialized = Array(repeating: 0, count: 5) // [0, 0, 0, 0, 0]
```

Arrays - Creating & Initializing



```
// 1. Literal Syntax
```

```
let primes = [2, 3, 5, 7, 11]
```

```
// 2. Array() Constructor
```

```
let vowels = Array(["a", "e", "i", "o", "u"])
```

```
// 3. Repeating Elements
```

```
let zeros = Array(repeating: 0, count: 10)
```

```
// 4. Range to Array
```

```
let rangeArray = Array(1...10) // [1, 2, 3, ..., 10]
```

```
// 5. From Other Collections
```

```
let set: Set = [1, 2, 3]
```

```
let arrayFromSet = Array(set) // Order not guaranteed!
```

```
// 6. Empty Arrays
```

```
var tasks: [String] = []
```

```
var scores = [Int]()
```


Arrays - Accessing Elements



```
let fruits = ["Apple", "Banana", "Cherry", "Date"]

// 1. Subscript Syntax
let firstFruit = fruits[0]    // "Apple"
let thirdFruit = fruits[2]    // "Cherry"

// 2. First & Last (Optional)
let first = fruits.first      // Optional("Apple")
let last = fruits.last        // Optional("Date")

// 3. Count & Checking
let count = fruits.count      // 4
let isEmpty = fruits.isEmpty  // false

// 4. Safe Access (Bounds checking)
if fruits.indices.contains(2) {
    print(fruits[2])          // Safe access
}

// 5. Slices (Creates new array)
let slice = fruits[1...2]     // ["Banana", "Cherry"]
```

Arrays - Modifying Elements



```
var shoppingList = ["Milk", "Eggs"]

// 1. Adding Elements
shoppingList.append("Bread")           // ["Milk", "Eggs", "Bread"]
shoppingList.insert("Butter", at: 1)    // ["Milk", "Butter", "Eggs", "Bread"]

// 2. Combining Arrays
shoppingList += ["Cheese", "Yogurt"]    // Appends multiple

// 3. Modifying Elements
shoppingList[0] = "Almond Milk"         // Replaces "Milk"

// 4. Removing Elements
let removed = shoppingList.remove(at: 1) // Removes "Butter"
shoppingList.removeFirst()               // Removes first
shoppingList.removeLast()                // Removes last
shoppingList.removeAll()                 // Clears array

// 5. Remove with condition
shoppingList.removeAll { $0.contains("Milk") }
```

Arrays - Iteration



```
let scores = [95, 87, 92, 78]
```

```
// 1. For-in Loop
```

```
for score in scores {  
    print("Score: \(score)")  
}
```

```
// 2. For-in with Index
```

```
for (index, score) in scores.enumerated() {  
    print("Student \(index + 1): \(score)")  
}
```

```
// 3. ForEach Method
```

```
scores.forEach { score in  
    print(score)  
}
```

```
// 4. While Loop
```

```
var i = 0  
while i < scores.count {  
    print(scores[i])  
    i += 1  
}
```

```
// 5. Using indices
```

```
for index in scores.indices {  
    print(scores[index])  
}
```

Arrays - Higher-Order Functions



```
let numbers = [1, 2, 3, 4, 5]
```

```
// 1. map - Transform each element
```

```
let squared = numbers.map { $0 * $0 } // [1, 4, 9, 16, 25]
```

```
let strings = numbers.map { "Number \($0)" }
```

```
// 2. filter - Select elements
```

```
let evens = numbers.filter { $0 % 2 == 0 } // [2, 4]
```

```
// 3. reduce - Combine to single value
```

```
let sum = numbers.reduce(0, +) // 15
```

```
let product = numbers.reduce(1, *) // 120
```

```
// 4. compactMap - Transform and remove nil
```

```
let maybeNumbers = ["1", "2", "three", "4"]
```

```
let validNumbers = maybeNumbers.compactMap { Int($0) } // [1, 2, 4]
```

```
// 5. flatMap - Flatten nested arrays
```

```
let nested = [[1, 2], [3, 4], [5]]
```

```
let flat = nested.flatMap { $0 } // [1, 2, 3, 4, 5]
```

Arrays - Searching & Sorting



```
var numbers = [5, 2, 8, 1, 9, 3]
```

```
// 1. Sorting
```

```
let sortedAsc = numbers.sorted()           // [1, 2, 3, 5, 8, 9]
```

```
let sortedDesc = numbers.sorted(by: >)    // [9, 8, 5, 3, 2, 1]
```

```
numbers.sort() // Sorts in place
```

```
// 2. Searching
```

```
let containsFive = numbers.contains(5)    // true
```

```
let firstEven = numbers.first { $0 % 2 == 0 } // Optional(2)
```

```
let lastOdd = numbers.last { $0 % 2 != 0 }  // Optional(3)
```

```
// 3. Finding Index
```

```
if let index = numbers.firstIndex(of: 8) {
```

```
    print("Found at index \(index)") // 2
```

```
}
```

```
// 4. Min/Max
```

```
let minValue = numbers.min() // Optional(1)
```

```
let maxValue = numbers.max() // Optional(9)
```

```
// 5. All/None satisfy condition
```

```
let allPositive = numbers.allSatisfy { $0 > 0 } // true
```

Sets



- **Characteristics:**
- Unordered (no guaranteed order)
- Unique values (no duplicates)
- Fast membership testing ($O(1)$)
- Value type (copied when assigned)

```
// Declaration
var uniqueNumbers: Set<Int> = [1, 2, 3, 4, 5]
var colors: Set = ["Red", "Green", "Blue"] // Type inferred
var emptySet: Set<String> = []
var anotherEmpty = Set<Int>()
```

Sets - Operations



```
let setA: Set = [1, 2, 3, 4, 5]
```

```
let setB: Set = [4, 5, 6, 7, 8]
```

```
// 1. Union - All elements from both
```

```
let union = setA.union(setB)           // {1, 2, 3, 4, 5, 6, 7, 8}
```

```
// 2. Intersection - Common elements
```

```
let intersection = setA.intersection(setB) // {4, 5}
```

```
// 3. Subtracting - In A but not B
```

```
let subtracting = setA.subtracting(setB)   // {1, 2, 3}
```

```
// 4. Symmetric Difference - Not in both
```

```
let symmetricDiff = setA.symmetricDifference(setB) // {1, 2, 3, 6, 7, 8}
```

```
// 5. Subset/Superset Checking
```

```
let smallSet: Set = [1, 2]
```

```
smallSet.isSubset(of: setA)           // true
```

```
setA.isSuperset(of: smallSet)         // true
```

```
setA.isDisjoint(with: setB)           // false (they share elements)
```

Sets - Methods & Properties



```
var fruits: Set = ["Apple", "Banana", "Orange"]
```

```
// 1. Adding Elements
```

```
fruits.insert("Mango")           // Returns (inserted: true, memberAfterInsert: "Mango")
```

```
fruits.insert("Apple")          // Returns (inserted: false, ...) – already exists
```

```
// 2. Removing Elements
```

```
fruits.remove("Banana")         // Returns "Banana" (or nil if not found)
```

```
fruits.removeFirst()           // Removes random element
```

```
fruits.removeAll()
```

```
// 3. Checking Membership
```

```
let hasApple = fruits.contains("Apple") // true
```

```
// 4. Count & Empty
```

```
let count = fruits.count        // Number of elements
```

```
let isEmpty = fruits.isEmpty    // false
```

```
// 5. Iteration (order not guaranteed)
```

```
for fruit in fruits {  
    print(fruit)  
}
```

```
// 6. Convert to Array (sorted)
```

```
let sortedFruits = fruits.sorted() // Returns Array
```


Dictionaries



- **Characteristics:**
- Key-Value pairs (unique keys)
- Unordered (Swift 4+)
- Keys must be Hashable
- Fast lookup by key ($O(1)$)

```
// Declaration
```

```
var scores: [String: Int] = ["Alice": 95, "Bob": 87, "Charlie": 92]
```

```
var capitals = ["France": "Paris", "Japan": "Tokyo"] // Type inferred
```

```
var emptyDict: [Int: String] = [:]
```

```
var anotherEmpty = Dictionary<String, Bool>()
```

Dictionaries - Creating & Accessing



```
var studentGrades = ["Alice": 95, "Bob": 87, "Charlie": 92]

// 1. Accessing Values
let aliceGrade = studentGrades["Alice"]           // Optional(95)
let davidGrade = studentGrades["David"]           // nil

// 2. Default Values
let grade = studentGrades["David", default: 0]    // 0 (not optional!)

// 3. Adding/Updating
studentGrades["David"] = 88                       // Adds new entry
studentGrades["Alice"] = 96                       // Updates existing

// 4. updateValue (returns old value)
let oldGrade = studentGrades.updateValue(97, forKey: "Bob") // Optional(87)

// 5. Removing
studentGrades["Charlie"] = nil                    // Removes entry
let removed = studentGrades.removeValue(forKey: "David") // Returns value

// 6. Count & Checking
let count = studentGrades.count                  // 2
let isEmpty = studentGrades.isEmpty              // false
```

Dictionaries - Iteration



```
let airportCodes = ["JFK": "New York", "LHR": "London", "CDG": "Paris"]
```

```
// 1. For-in Loop (key-value tuples)
```

```
for (code, city) in airportCodes {  
    print("\(code): \(city)")  
}
```

```
// 2. Keys only
```

```
for code in airportCodes.keys {  
    print("Airport code: \(code)")  
}
```

```
// 3. Values only
```

```
for city in airportCodes.values {  
    print("City: \(city)")  
}
```

```
// 4. Convert to Arrays
```

```
let codes = Array(airportCodes.keys)    // ["JFK", "LHR", "CDG"]  
let cities = Array(airportCodes.values) // ["New York", "London", "Paris"]
```

```
// 5. Sorted iteration
```

```
for (code, city) in airportCodes.sorted(by: { $0.key < $1.key }) {  
    print("\(code): \(city)")  
}
```

Tuples



```
// Basic Tuple
let http404Error = (404, "Not Found") // Type: (Int, String)

// Named Tuple (Recommended)
let person = (name: "Alice", age: 30, isStudent: false)

// Accessing Elements
let statusCode = http404Error.0 // 404 (by position)
let statusMessage = http404Error.1 // "Not Found"

let userName = person.name // "Alice" (by name)
let userAge = person.age // 30

// Type Aliases for Clarity
typealias Coordinate = (x: Double, y: Double)
let location: Coordinate = (40.7128, -74.0060)

// Single-element tuples need comma
let single = ("hello",) // Type: (String,)
```

Tuples - Decomposition & Comparison



```
// 1. Decomposition
let (code, message) = http404Error
print("Code: \(code), Message: \(message)")

// 2. Ignoring parts
let (justCode, _) = http404Error      // Only get first value

// 3. In Functions (Multiple Returns)
func minMax(array: [Int]) -> (min: Int, max: Int) {
    var currentMin = array[0]
    var currentMax = array[0]
    for value in array {
        if value < currentMin { currentMin = value }
        if value > currentMax { currentMax = value }
    }
    return (currentMin, currentMax)
}

// 4. Tuple Comparison (element-wise)
(1, "apple") < (2, "banana")    // true (compares 1 < 2)
(3, "apple") < (3, "bird")      // true (3 == 3, compares "apple" < "bird")
(4, "dog") == (4, "dog")        // true
```

Collection Comparison



■ WHEN TO USE EACH

■ Use ARRAY when:

- - Order matters
- - Need indexed access
- - Duplicates allowed
- - Example: Shopping list, playlist

■ Use SET when:

- - Need unique values
- - Order doesn't matter
- - Fast membership testing needed
- - Mathematical set operations
- - Example: Unique tags, visited pages

■ Use DICTIONARY when:

- - Key-value associations
- - Fast lookup by key
- - Example: User profiles, settings, caches

■ Use TUPLE when:

- - Fixed number of related values
- - Temporary grouping
- - Multiple return values from function
- - Example: Coordinates, HTTP response



Object Oriented Programming



What is Object-Oriented Programming?

- Programming Paradigm Based on "Objects"
- Objects bundle data and behavior together
- **Four Pillars of OOP**
 - **Encapsulation** - Hide implementation details
 - **Inheritance** - Create new classes from existing ones
 - **Polymorphism** - Different forms through interfaces
 - **Abstraction** - Simplify complex reality
- **Why OOP in Swift?**
 - iOS/macOS frameworks are OOP-based (UIKit, AppKit)
 - Better code organization
 - Reusability and maintainability
 - Natural modeling of real-world concepts

Classes vs Structs



```
// STRUCT (Value Type)
struct Point {
    var x: Double
    var y: Double

    func distance(to other: Point) -> Double {
        let deltaX = x - other.x
        let deltaY = y - other.y
        return sqrt(deltaX * deltaX + deltaY * deltaY)
    }
}

// CLASS (Reference Type)
class Vehicle {
    var speed: Double = 0
    var fuelLevel: Double = 100

    func drive(distance: Double) {
        // Implementation
    }
}
```

Aspect	Struct	Class
Type	Value Type	Reference Type
Inheritance	Not supported	Supported
Deinitializer	No	Yes (deinit)
Copy Behavior	Copy on assignment	Reference on assignment
Mutability	Must use mutating	Methods can modify properties
Memory	Stack (usually)	Heap
Thread Safety	Safer (copies)	Risk of race conditions

Class vs Struct Decision Guide



// USE STRUCTS WHEN:

// 1. Simple data values

```
struct Point {  
    var x, y: Double  
}
```

// 2. Copy semantics desired

```
var p1 = Point(x: 0, y: 0)  
var p2 = p1 // Copy happens  
p2.x = 5    // p1.x still 0
```

// 3. No inheritance needed

// 4. Thread safety important

// 5. Small, simple types

// USE CLASSES WHEN:

// 1. Reference semantics needed

```
class FileHandle {  
    var file: UnsafeMutablePointer<FILE>  
    // Multiple parts of code need same file  
}
```

// 2. Inheritance needed

```
class ViewController: UIViewController { }
```

// 3. Objective-C interoperability

```
@objc class MyClass: NSObject { }
```

// 4. Identity matters (=== operator)

```
let obj1 = MyClass()  
let obj2 = obj1  
print(obj1 === obj2) // true
```

// 5. Deinitialization needed

```
class Database
```

Defining Classes in Swift



```
class Person {
    // 1. PROPERTIES (Stored Properties)
    var name: String
    var age: Int
    var email: String?

    // 2. COMPUTED PROPERTIES
    var isAdult: Bool {
        return age >= 18
    }

    // 3. PROPERTY OBSERVERS
    var score: Int = 0 {
        willSet {
            print("Score will change from \(score) to \(newValue)")
        }
        didSet {
            print("Score changed from \(oldValue) to \(score)")
        }
    }
}
```

```
// 4. LAZY PROPERTIES (Initialized when first accessed)
lazy var bio: String = {
    return "\(name), \(age) years old"
}()

// 5. INITIALIZER (Constructor)
init(name: String, age: Int, email: String? = nil) {
    self.name = name
    self.age = age
    self.email = email
}

// 6. METHODS
func introduce() {
    print("Hello, I'm \(name) and I'm \(age) years old.")
}

func haveBirthday() {
    age += 1
    print("Happy Birthday! Now I'm \(age)")
}

// 7. DEINITIALIZER (Destructor)
deinit {
    print("\(name) instance is being deallocated")
}
}
```

Creating & Using Objects



```
// 1. CREATING INSTANCES (Objects)
let person1 = Person(name: "Alice", age: 25)
let person2 = Person(name: "Bob", age: 17, email: "bob@email.com")

// 2. ACCESSING PROPERTIES
print(person1.name)           // "Alice"
print(person1.isAdult)        // true (computed property)
person1.score = 100           // Triggers property observers

// 3. CALLING METHODS
person1.introduce()            // "Hello, I'm Alice and I'm 25 years old."
person1.haveBirthday()         // "Happy Birthday! Now I'm 26"

// 4. MODIFYING PROPERTIES
person1.name = "Alice Smith"
person1.score += 50

// 5. REFERENCE BEHAVIOR (Classes only)
let person3 = person1          // person3 references SAME object as person1
person3.age = 30
print(person1.age)             // 30 (both reference same object)

// 6. COMPARING OBJECTS
let person4 = Person(name: "Alice", age: 25)
print(person1 === person4)     // false (different objects)
print(person1 === person3)     // true (same object)
```

Initializers - Constructing Objects Properly



```
class Product {  
    let id: String  
    var name: String  
    var price: Double  
  
    // 1. DESIGNATED INITIALIZER (Primary)  
    init(id: String, name: String, price: Double) {  
        self.id = id  
        self.name = name  
        self.price = price  
    }  
  
    // 2. CONVENIENCE INITIALIZER (Secondary)  
    convenience init(name: String, price: Double) {  
        let id = UUID().uuidString  
        self.init(id: id, name: name, price: price)  
    }  
}
```

```
    // 3. FAILABLE INITIALIZER (Can return nil)  
    init?(dictionary: [String: Any]) {  
        guard let id = dictionary["id"] as? String,  
              let name = dictionary["name"] as? String,  
              let price = dictionary["price"] as? Double else {  
            return nil  
        }  
        self.id = id  
        self.name = name  
        self.price = price  
    }  
  
    // 4. REQUIRED INITIALIZER (Must be implemented by subclasses)  
    required init() {  
        self.id = UUID().uuidString  
        self.name = "Unknown"  
        self.price = 0.0  
    }  
}
```

Inheritance in Swift



```
// BASE CLASS (Parent/Super class)
class Animal {
    var name: String
    var sound: String

    init(name: String, sound: String) {
        self.name = name
        self.sound = sound
    }

    func makeSound() {
        print("\(name) says \(sound)!")
    }

    func eat() {
        print("\(name) is eating")
    }
}
```

```
// DERIVED CLASS (Child/Sub class)
class Dog: Animal {
    var breed: String

    // Override initializer
    init(name: String, breed: String) {
        self.breed = breed
        super.init(name: name, sound: "Woof")
    }

    // Override method
    override func makeSound() {
        print("\(name) the \(breed) barks: \(sound)!")
    }

    // Add new method
    func fetch(item: String) {
        print("\(name) fetches the \(item)")
    }
}
```

Access Control - Controlling Visibility and Encapsulation



```
class BankAccount {  
    // PRIVATE - Only within this class  
    private var accountNumber: String  
    private var pin: String  
  
    // INTERNAL - Default (accessible within module)  
    internal var balance: Double  
  
    // PUBLIC - Accessible from anywhere  
    public let accountHolder: String  
  
    // FILEPRIVATE - Accessible within same file  
    fileprivate var transactionHistory: [String] = []  
  
    // OPEN - Like public, but can be subclassed outside module  
    open var interestRate: Double = 0.02  
  
    init(accountHolder: String, accountNumber: String, pin: String) {  
        self.accountHolder = accountHolder  
        self.accountNumber = accountNumber  
        self.pin = pin  
        self.balance = 0.0  
    }  
}
```

```
// PUBLIC METHODS (Interface)  
public func deposit(amount: Double) {  
    balance += amount  
    logTransaction("Deposited: ${amount}")  
}  
  
// PRIVATE HELPER METHOD  
private func logTransaction(_ description: String) {  
    transactionHistory.append("\(Date()): \(description)")  
}  
  
// INTERNAL HELPER  
internal func getTransactionCount() -> Int {  
    return transactionHistory.count  
}  
  
// USAGE  
let account = BankAccount(accountHolder: "John Doe",  
                           accountNumber: "123456",  
                           pin: "1234")  
  
account.deposit(amount: 1000)           // OK - public  
// account.balance = 5000               // OK - internal (same module)  
// account.accountNumber = "999999"     // ERROR - private  
// account.logTransaction("Test")       // ERROR - private
```

Access Controls



- **private**
 - **Scope:** Accessible only within the enclosing declaration. Most restrictive.
 - **Use when:** Implementation detail should never be accessed outside
- **fileprivate**
 - **Scope:** Accessible anywhere in the same file
 - **Use when:** Multiple types/extensions in same file need access
- **internal (Default)**
 - **Scope:** Accessible within the same module. Default access level in Swift.
- **public**
 - **Scope:** Accessible outside the module. Cannot be subclassed or overridden outside module.
- **open**
 - **Scope:** Same as public. Allows subclassing and overriding outside the module.

Protocol



- A **protocol** is a **blueprint of requirements**:
 - Properties
 - Methods
 - Initializers
- Any class, struct, or enum that conforms to a protocol **must implement those requirements**.
- Similar to interface in Java
- Protocols solve problems that inheritance cannot:
 - Multiple inheritance
 - Loose coupling
 - Testability
 - Reusability
 - Clean architecture

Protocol

```
// Define a protocol
protocol Vehicle {
    var brand: String { get }
    var currentSpeed: Double { get set }

    func startEngine()
    func stopEngine()
    func accelerate(by amount: Double)
    func brake(by amount: Double)
}
```



```
// Conform to the protocol with a struct
struct Car: Vehicle {
    let brand: String
    var currentSpeed: Double = 0
    var gear: Int = 1

    func startEngine() {
        print("\(brand) engine started!")
    }

    func stopEngine() {
        print("\(brand) engine stopped.")
    }

    func accelerate(by amount: Double) {
        currentSpeed += amount
        print("\(brand) accelerating to \(currentSpeed) km/h")
    }

    func brake(by amount: Double) {
        currentSpeed = max(0, currentSpeed - amount)
        print("\(brand) braking to \(currentSpeed) km/h")
    }
}
```